

Michal Hornicky

SENIOR SOFTWARE ENGINEER · AGENT INFRASTRUCTURE · ML SYSTEMS

Zurich, Switzerland

☎ +41 79 810 28 94

✉ semtexzv@gmail.com

🏠 <https://github.com/semtexzv>

🌐 [semtexzv](#)



About Me

Right now I'm focused on one problem: recursive self-improvement isn't a model problem — it's an environment problem, the tooling and harness and whole ecosystem around the model. I can't train a frontier model alone; what I can build is the self-improving developer environments that drive those models to their maximum effectiveness — at Google DeepMind on the Gemini team by day, on a fleet of autonomous-agent systems by night, where the engineering process generates the data that improves first the harness, then the model. I'm goal-oriented and prioritize developer velocity; my process is built on fast idea exploration — prototypes, design docs, and collaborative discussion. Ten years across ML systems, distributed infrastructure, and large-scale applications. I ship self-modifying agents gated, sandboxed, and fully auditable — because I think that's the only responsible way to build them.

Skills

Agent Systems	Autonomous coding agents, Multi-agent orchestration, Self-improving loops, LLM harness design, Tool calling, MCP protocol
ML / AI	PyTorch, MLX, Training pipelines, Distributed training, Reinforcement learning, Transformer architectures, Autodiff
Systems	Distributed systems, HPC, GPGPU (CUDA/OpenCL), ML Accelerators, Hermetic builds (Buck2/Bazel), Sandboxing, Actor models
Infrastructure	Kubernetes, Terraform, Large-scale data processing, Observability, CI/CD, GCP
Languages	Python, Rust, Go, C/C++, Swift, Kotlin, TypeScript
Spoken	Slovak (Native), Czech (Native), English (Fluent), German (Beginner)

Work Experience

Senior Software Engineer — Gemini / Google DeepMind

Zurich

GOOGLE

Nov 2024 – Present

- Lead a small team building agent-native developer infrastructure for the Gemini organization — fleets of collaborating agents that take over significant parts of the engineering loop: planning, implementation, and review
- Pioneered agent integration across the Gemini application stack: multi-agent orchestration, tool-augmented workflows, and the internal best practices around them
- Drive adoption through tooling and workshops — the function is researcher and engineer velocity: make the people around me faster by giving them better machinery
- Implemented core product features in the Gemini mobile application

Software Engineer — Assistant & Gemini

Zurich

GOOGLE

Nov 2021 – Nov 2024

- Developed automated testing architecture and tooling for Android and iOS applications at scale
- Supported testing and releases for Google Assistant, Search, and Gemini/Bard mobile apps
- Pioneered use of LLMs for automated testing and software verification, building early internal tools for AI-assisted development

Senior Software Engineer / Team Lead

Brno

RED HAT

Jul 2020 – Sep 2021

- Led a team of 5 engineers building the Patch application on cloud.redhat.com
- Engineered a custom database and evaluation engine for RHEL package dependency resolution, overcoming PostgreSQL performance limits — required novel algorithmic approaches to graph traversal at scale
- Scaled the platform to serve thousands of large-scale RHEL fleet deployments daily

Software Engineer

Brno

RED HAT

Sep 2019 – Jul 2020

- Initiated development of the Patch application; built Go libraries for integrating into Python ecosystem

- Full lifecycle management of 10+ B2B Android applications; led architectural modernization to Kotlin, MVVM, reactive dataflow

AI Projects

Bedrock — AI-Native Software Company in a Box

A SELF-IMPROVING MULTI-AGENT COLLECTIVE THAT BUILDS, REVIEWS, AND IMPROVES A SOFTWARE PLATFORM END TO END

SEMTEXZV.GITHUB.IO/BEDROCK

2026 – Present

- A self-improving multi-agent collective — effectively “Kubernetes for agents”: an orchestration layer where a swarm of collaborating coding agents (≈ 15 today) continuously build, review, and integrate changes into a shared, self-building monorepo, end to end
- Built on Buck2 and jj — a self-integrating monorepo with automatic CI and a web review front end, so every change an agent makes is reproducible and lands through one consistent pipeline
- Generates its own data: every change, dispatch, agent conversation, and review is a logged trajectory. I’m building it toward a self-researching environment that creates its own training data and fine-tunes its own agents — the system producing the data that makes its next generation better. A Voyager-style lifelong-learning loop applied to a software platform
- Bounded self-modification: a structured security layer protects the system’s core, so agents can evolve the platform but can’t make destructive or self-negative changes to its foundations. Changes to the protected core require human approval, and the automated code reviewer is explicitly advisory — never the sole authority on an adversarial change. Safety is a structural property, not a policy

llm-lab — Exploratory LLM Training

TRAINING AND EVALUATING LANGUAGE MODELS ON APPLE SILICON

 SEMTEXZV/LLM-LAB

2025 – Present

- Complete 150M-parameter training/evaluation pipeline (MLX): custom optimizers, mixed precision, IsoFLOP comparison harness for controlled architecture comparisons
- I reimplement current frontier architectures from the papers — linear-attention variants, attention-residual schemes — to keep a working feel for what the researchers I’d support actually do

mage — Self-Recompiling Coding Agent

AN AGENT THAT EXTENDS ITSELF BY RECOMPILING ITS OWN BINARY

 SEMTEXZV/MAGE

- Embeds a snapshot of its own source; splices in new modules, recompiles a fresh binary, and hot-swaps into the next generation via a supervising monitor — self-modification at the compiled-artifact level
- Rust workspace (edition 2024): concurrent/serial tool dispatch, hierarchical cancellation, differential terminal renderer

Termineer — Autonomous AI Coding Agent

SELF-DIRECTED CODING AGENT BUILT ON A CONFIGURABLE TOOL-CALLING GRAMMAR

 SEMTEXZV/TERMINEER

- A configurable grammar unified function calling across Anthropic, Google, and OpenAI — built when providers had no shared format for it
- Autonomous task decomposition with specialized sub-agents; MCP protocol support

GenAgent — Always-On Personal Assistant

TASKBAR AI ASSISTANT (MACOS/IOS), EXPLORED ALONGSIDE TERMINEER

 SEMTEXZV/GENAGENT

- Menu-bar assistant invocable anytime for coding and general help — the personal-assistant thread, explored before and alongside Termineer’s coding agent

Distributed Computing Infrastructure

Rube — Generalized Control-Loop Architecture

 SEMTEXZV/RUBE

- A reconciliation/control-loop runtime for arbitrary resources — “Kubernetes without the YAML or containers”

Protokit — High-Performance Protocol Buffers for Rust

 SEMTEXZV/PROKIT

- Only Rust implementation supporting full TextFormat specification; competitive with prost; dual front-end compiler

Stor — Embedded Key-Value Database

 SEMTEXZV/STOR

- Typed key-value database on RocksDB with zero-copy deserialization; storage engine for a low-latency cryptocurrency portfolio platform

Education

Master's Degree, Information Technology

BRNO UNIVERSITY OF TECHNOLOGY

Thesis: Distributed algorithmic cryptocurrency trading system (Rust, ZeroMQ, NATS, Kubernetes)

Brno

2017 – 2019

Bachelor's Degree, Information Technology

BRNO UNIVERSITY OF TECHNOLOGY

Brno

2014 – 2017